MATH 128 - HW6 - Jie Wen

Problem 1: Problems from the textbook, chapter 2.3

2

$f(x) = -x^3 - \cos x$, $f'(x) = -3x^2 + \sin x$.

Iterations from $p_0 = -1$

- $f(-1) = 1 - \cos 1 \approx 0.459697694$
  $f'(-1) = -3 - \sin 1 \approx -3.841470985$
  $p_1 = -1 - \frac{0.459697694}{-3.841470985} \approx -0.8803328996$
- $f(-0.8803328996) \approx 0.0453511546$
  $f'(-0.8803328996) \approx -3.095908986$
  $p_2 = -0.8803328996 - \frac{0.0453511546}{-3.095908986} \approx -0.8656841632$

Could $p_0 = 0$ be used?

No. $f'(0) = \sin 0 - 3 \cdot 0^2 = 0$, so the Newton update $p_1 = p_0 - \frac{f(p_0)}{f'(p_0)}$ is undefined (division by zero). Newton's method assumes a nonzero derivative at the iterate; when the derivative is zero (or very small), the method can fail.

6

MATLAB codes:

```
%% newton_batch.m
% Newton's method for a batch of scalar equations (problems a–f).
% Tolerance: 1e-5; Max iterations: 100
% Requires MATLAB R2016b+ to allow local functions in scripts.

format long g; clc;

TOL   = 1e-5;
MAXIT = 100;

fprintf('=== Newton''s Method (TOL = %g) ===\n\n', TOL);

%% (a) e^x + 2^{-x} + 2 cos x − 6 = 0 on [1, 2]
f  = @(x) exp(x) + 2.^(-x) + 2*cos(x) - 6;
df = @(x) exp(x) - log(2).*2.^(-x) - 2*sin(x);          % d/dx (2^{-x}) = -ln(2)*2^{-x}
x0 = mean([1, 2]);
xa = newton(f, df, x0, TOL, MAXIT);
report('(a) [1,2]', xa, f(xa));

%% (b) ln(x-1) + cos(x-1) = 0 on [1.3, 2]
f  = @(x) log(x-1) + cos(x-1);
df = @(x) 1./(x-1) - sin(x-1);
x0 = mean([1.3, 2]);
xb = newton(f, df, x0, TOL, MAXIT);
report('(b) [1.3,2]', xb, f(xb));

%% (c) 2x cos(2x) − (x−2)^2 = 0
% on [2,3]
f  = @(x) 2*x.*cos(2*x) - (x-2).^2;
df = @(x) 2*cos(2*x) - 4*x.*sin(2*x) - 2*(x-2);
x0 = mean([2, 3]);
xc1 = newton(f, df, x0, TOL, MAXIT);
report('(c) [2,3]', xc1, f(xc1));
% on [3,4]
x0 = mean([3, 4]);
xc2 = newton(f, df, x0, TOL, MAXIT);
report('(c) [3,4]', xc2, f(xc2));

%% (d) (x−2)^2 − ln x = 0
% on [1,2]
f  = @(x) (x-2).^2 - log(x);
df = @(x) 2*(x-2) - 1./x;
x0 = mean([1, 2]);
xd1 = newton(f, df, x0, TOL, MAXIT);
report('(d) [1,2]', xd1, f(xd1));
% on [e,4]
x0 = mean([exp(1), 4]);
xd2 = newton(f, df, x0, TOL, MAXIT);
report('(d) [e,4]', xd2, f(xd2));
```

```
%% (e) e^x - 3x^2 = 0
% on [0,1]
f  = @(x) exp(x) - 3*x.^2;
df = @(x) exp(x) - 6*x;
x0 = mean([0, 1]);
xe1 = newton(f, df, x0, TOL, MAXIT);
report('(e) [0,1]', xe1, f(xe1));
% on [3,5]
x0 = mean([3, 5]);
xe2 = newton(f, df, x0, TOL, MAXIT);
report('(e) [3,5]', xe2, f(xe2));

%% (f) sin x - e^{-x} = 0
f  = @(x) sin(x) - exp(-x);
df = @(x) cos(x) + exp(-x);       % d/dx(-e^{-x}) = +e^{-x}
% on [0,1]
x0 = mean([0, 1]);
xf1 = newton(f, df, x0, TOL, MAXIT);
report('(f) [0,1]', xf1, f(xf1));
% on [3,4]
x0 = mean([3, 4]);
xf2 = newton(f, df, x0, TOL, MAXIT);
report('(f) [3,4]', xf2, f(xf2));
% on [6,7]
x0 = mean([6, 7]);
xf3 = newton(f, df, x0, TOL, MAXIT);
report('(f) [6,7]', xf3, f(xf3));

fprintf('\nDone.\n');

%% ---------- Local functions ----------

function x = newton(f, df, x0, tol, maxit)
%NEWTON  Scalar Newton's method with |x_{k+1}-x_k| stopping.
%   x = NEWTON(f, df, x0, tol, maxit)
%   - f, df : function handles for f(x) and f'(x)
%   - x0    : initial guess
%   - tol   : stopping tolerance on step size
%   - maxit : maximum iterations
%
%   Stops when |x_{k+1} - x_k| < tol or derivative is ~0.

    x  = x0;
    for k = 1:maxit
        fx  = f(x);
        dfx = df(x);
        if ~isfinite(fx) || ~isfinite(dfx)
            error('Non-finite f/df at x = %.16g', x);
        end
        if abs(dfx) < eps^(2/3)   % simple guard against tiny derivative
            warning('Derivative near zero at x = %.16g; aborting.', x);
            return;
        end
        xnew = x - fx/dfx;
        if ~isfinite(xnew)
            error('Non-finite iterate produced from x = %.16g', x);
        end
        if abs(xnew - x) < tol
            x = xnew;
            return;
        end
        x = xnew;
    end
    warning('Max iterations (%d) reached; last x = %.16g', maxit, x);
end

function report(tag, x, fx)
    fprintf('%-10s  root ≈ %.10f   residual f(x) = %.3e\n', tag, x, fx);
end
```

Outputs:

```
=== Newton's Method (TOL = 1e-05) ===

(a) [1,2]   root ≈ 1.8293836019   residual f(x) = 8.882e-16
(b) [1.3,2]  root ≈ 1.3977484759   residual f(x) = -1.006e-10
(c) [2,3]   root ≈ 2.3706869177   residual f(x) = 2.248e-12
(c) [3,4]   root ≈ 3.7221127731   residual f(x) = -7.867e-11
(d) [1,2]   root ≈ 1.4123911720   residual f(x) = 1.665e-16
(d) [e,4]   root ≈ 3.0571035500   residual f(x) = 2.154e-13
(e) [0,1]   root ≈ 0.9100075725   residual f(x) = -1.377e-14
(e) [3,5]   root ≈ 3.7330790287   residual f(x) = 4.245e-10
```

```
(f) [0,1]   root ≈ 0.58853274440   residual f(x) = -6.161e-12
(f) [3,4]   root ≈ 3.0963639324    residual f(x) = -1.041e-16
(f) [6,7]   root ≈ 6.2850492734    residual f(x) = 1.427e-16

Done.
```

8

MATLAB codes:

```
%% secant_batch.m
% Secant method for the Exercise 6 equations (parts a–f).
% Tolerance: 1e-5; Max iterations: 200
% MATLAB R2016b+ (uses local functions in a script).

format long g; clc;

TOL   = 1e-5;
MAXIT = 200;

fprintf('=== Secant Method (TOL = %g) ===\n\n', TOL);

%% (a) e^x + 2^{-x} + 2 cos x − 6 = 0 on [1, 2]
f = @(x) exp(x) + 2.^(-x) + 2*cos(x) - 6;
xa = secant(f, 1, 2, TOL, MAXIT);
report('(a) [1,2]', xa, f(xa));

%% (b) ln(x-1) + cos(x-1) = 0 on [1.3, 2]
f = @(x) log(x-1) + cos(x-1);
xb = secant(f, 1.3, 2.0, TOL, MAXIT);
report('(b) [1.3,2]', xb, f(xb));

%% (c) 2x cos(2x) − (x-2)^2 = 0
f = @(x) 2*x.*cos(2*x) - (x-2).^2;
% on [2,3]
xc1 = secant(f, 2.0, 3.0, TOL, MAXIT);
report('(c) [2,3]', xc1, f(xc1));
% on [3,4]
xc2 = secant(f, 3.0, 4.0, TOL, MAXIT);
report('(c) [3,4]', xc2, f(xc2));

%% (d) (x-2)^2 − ln x = 0
f = @(x) (x-2).^2 - log(x);
% on [1,2]
xd1 = secant(f, 1.0, 2.0, TOL, MAXIT);
report('(d) [1,2]', xd1, f(xd1));
% on [e,4]
xd2 = secant(f, exp(1), 4.0, TOL, MAXIT);
report('(d) [e,4]', xd2, f(xd2));

%% (e) e^x − 3x^2 = 0
f = @(x) exp(x) - 3*x.^2;
% on [0,1]
xe1 = secant(f, 0.0, 1.0, TOL, MAXIT);
report('(e) [0,1]', xe1, f(xe1));
% on [3,5]
xe2 = secant(f, 3.0, 5.0, TOL, MAXIT);
report('(e) [3,5]', xe2, f(xe2));

%% (f) sin x − e^{-x} = 0
f = @(x) sin(x) - exp(-x);
% on [0,1]
xf1 = secant(f, 0.0, 1.0, TOL, MAXIT);
report('(f) [0,1]', xf1, f(xf1));
% on [3,4]
xf2 = secant(f, 3.0, 4.0, TOL, MAXIT);
report('(f) [3,4]', xf2, f(xf2));
% on [6,7]
xf3 = secant(f, 6.0, 7.0, TOL, MAXIT);
report('(f) [6,7]', xf3, f(xf3));

fprintf('\nDone.\n');

%% ---------- Local functions ----------

function x = secant(f, p0, p1, tol, maxit)
%SECANT  Scalar secant method with |p_{k+1}-p_k| stopping rule.
%   x = SECANT(f, p0, p1, tol, maxit)
%   Implements Algorithm 2.4; guards against division by near-zero slopes.
```

```
    q0 = f(p0);
    q1 = f(p1);

    for k = 2:maxit
        denom = (q1 - q0);
        if ~isfinite(q0) || ~isfinite(q1)
            error('Non-finite function value (q0/q1).');
        end
        if abs(denom) <= eps^(2/3)
            % Small slope; nudge the older point slightly
            p0 = p0 + max(1, abs(p0))*1e-12;
            q0 = f(p0);
            denom = (q1 - q0);
            if abs(denom) <= eps^(2/3)
                warning('Secant slope ~0 at iteration %d; aborting.', k);
                x = p1; return;
            end
        end

        p = p1 - q1*(p1 - p0)/denom;    % Algorithm 2.4, Step 3
        if ~isfinite(p)
            error('Non-finite iterate produced.');
        end

        if abs(p - p1) < tol           % Step-size stopping (Algorithm 2.4, Step 4)
            x = p; return;
        end

        % Update (Algorithm 2.4, Step 6)
        p0 = p1;  q0 = q1;
        p1 = p;   q1 = f(p1);
    end
    warning('Max iterations (%d) reached; returning last iterate.', maxit);
    x = p1;
end

function report(tag, x, fx)
    fprintf('%-12s root ≈ %.10f   residual f(x) = %.3e\n', tag, x, fx);
end
```

Outputs:

```
=== Secant Method (TOL = 1e-05) ===

(a) [1,2]    root ≈ 1.8293836019   residual f(x) = 2.307e-11
(b) [1.3,2]  root ≈ 1.3977484760   residual f(x) = -2.377e-12
(c) [2,3]    root ≈ 2.3706869177   residual f(x) = 3.376e-13
(c) [3,4]    root ≈ 3.7221127734   residual f(x) = -5.195e-09
(d) [1,2]    root ≈ 1.4123911720   residual f(x) = -1.742e-13
(d) [e,4]    root ≈ 3.0571035500   residual f(x) = 6.802e-11
(e) [0,1]    root ≈ 0.9100075715   residual f(x) = 2.827e-09
(e) [3,5]    root ≈ 3.7330790313   residual f(x) = 5.217e-08
(f) [0,1]    root ≈ 0.5885327423   residual f(x) = -2.266e-09
(f) [3,4]    root ≈ 3.0963639324   residual f(x) = -1.993e-11
(f) [6,7]    root ≈ 6.2850492734   residual f(x) = 2.226e-13

Done.
```

14

MATLAB codes:

```
%% root_tanpi_minus6.m
% Solve f(x) = tan(pi*x) - 6 near x ~ (1/pi)*atan(6) with:
%   (a) Bisection on [0, 0.48]
%   (c) Secant with p0 = 0, p1 = 0.48
%
% Exactly 10 iterations each. Prints tables and a brief summary.
% MATLAB fprintf formatting reference: see MathWorks docs.
% (e.g., https://www.mathworks.com/help/matlab/ref/fprintf.html)

clear; clc; format long g

% Problem setup
f       = @(x) tan(pi*x) - 6;            % target function
alpha   = atan(6)/pi;                    % "true" root (for error reporting)
a0      = 0.00;
b0      = 0.48;
```

```
p0      = 0.00;
p1      = 0.48;
Niter   = 10;

%% (a) Bisection method on [0, 0.48], 10 iterations
[a_hist, b_hist, c_hist, fc_hist] = bisection_fixed_iters(f, a0, b0, Niter);

% Report
fprintf('=== Bisection (N = %d) on [%.6f, %.6f] ===\n', Niter, a0, b0);
fprintf('%3s %14s %20s %18s %18s\n', 'k', 'c_k', 'f(c_k)', '|c_k - alpha|', 'bracket length');
for k = 1:Niter
    c   = c_hist(k);
    fc  = fc_hist(k);
    err = abs(c - alpha);
    len = b_hist(k) - a_hist(k);
    fprintf('%3d %14.10f %20.12e %18.10e %18.10e\n', k, c, fc, err, len);
end
c_bisect = c_hist(end);
fprintf('Bisection final approx: %.12f,  |err| = %.3e\n\n', c_bisect, abs(c_bisect - alpha));

%% (c) Secant method with p0 = 0, p1 = 0.48, 10 iterations
[p_hist, fp_hist] = secant_fixed_iters(f, p0, p1, Niter);

% Report
fprintf('=== Secant (N = %d) starting p0=%.6f, p1=%.6f ===\n', Niter, p0, p1);
fprintf('%3s %14s %20s %18s\n', 'k', 'p_k', 'f(p_k)', '|p_k - alpha|');
for k = 1:Niter
    pk  = p_hist(k);
    fpk = fp_hist(k);
    err = abs(pk - alpha);
    fprintf('%3d %14.10f %20.12e %18.10e\n', k, pk, fpk, err);
end
p_secant = p_hist(end);
fprintf('Secant final approx:   %.12f,  |err| = %.3e\n\n', p_secant, abs(p_secant - alpha));

%% Quick verdict
fprintf('=== Verdict ===\n');
fprintf('True root alpha ~= %.12f\n', alpha);
fprintf('Bisection (robust bracket) reached |err| = %.3e after %d steps.\n',

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% Local functions
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

function [a_hist, b_hist, c_hist, fc_hist] = bisection_fixed_iters(f, a, b, N)
    % Assumes f is continuous on [a,b] with f(a)*f(b) < 0
    % (For this problem: f(0) = -6 < 0 and f(0.48) > 0.)
    fa = f(a); fb = f(b);
    if ~isfinite(fa) || ~isfinite(fb) || fa*fb >= 0
        error('Bisection precondition failed: need finite f(a), f(b) with opposite signs.');
    end

    a_hist = zeros(N,1);
    b_hist = zeros(N,1);
    c_hist = zeros(N,1);
    fc_hist= zeros(N,1);

    for k = 1:N
        c  = (a + b)/2;
        fc = f(c);

        % Store history
        a_hist(k) = a;
        b_hist(k) = b;
        c_hist(k) = c;
        fc_hist(k)= fc;

        % Standard bisection step
        if fa*fc <= 0
            b  = c;
            fb = fc;
        else
            a  = c;
            fa = fc;
        end
    end
end

function [p_hist, fp_hist] = secant_fixed_iters(f, p0, p1, N)
    % Secant method: p_{k+1} = p_k - f(p_k)*(p_k - p_{k-1})/(f(p_k)-f(p_{k-1}))
    % Exactly N iterations recorded as p_1 ... p_N (p_1 = first update).
    % Includes simple guards against division by ~0 or Inf/NaN explosions.
    p_hist = nan(N,1);
    fp_hist= nan(N,1);
```

```
    q0 = f(p0);
    q1 = f(p1);

    for k = 1:N
        denom = (q1 - q0);
        if ~isfinite(denom) || abs(denom) < eps
            % If the secant slope is unusable, record and stop iterating.
            p  = NaN;
            qp = NaN;
        else
            p  = p1 - q1*(p1 - p0)/denom;
            qp = f(p);
        end

        p_hist(k)  = p;
        fp_hist(k) = qp;

        % Update for next step if possible
        p0 = p1;  q0 = q1;
        p1 = p;   q1 = qp;

        % If we hit NaN/Inf, continue to record but won't recover.
        if ~isfinite(p) || ~isfinite(qp)
            % keep looping to fill N rows (as requested), but values will be NaN/Inf
            % (No break to strictly keep N recorded iterations.)
        end
    end
end
```

Outputs:

```
=== Bisection (N = 10) on [0.000000, 0.480000] ===
  k            c_k               f(c_k)      |c_k - alpha|     bracket length
  1   0.2400000000  -5.060937494183e+00   2.0743154329e-01   4.8000000000e-01
  2   0.3600000000  -3.874891826843e+00   8.7431543289e-02   2.4000000000e-01
  3   0.4200000000  -2.105257145070e+00   2.7431543289e-02   1.2000000000e-01
  4   0.4500000000   3.137515146750e-01   2.5684567113e-03   6.0000000000e-02
  5   0.4350000000  -1.171182647807e+00   1.2431543289e-02   3.0000000000e-02
  6   0.4425000000  -5.245211508219e-01   4.9315432887e-03   1.5000000000e-02
  7   0.4462500000  -1.343497628774e-01   1.1815432887e-03   7.5000000000e-03
  8   0.4481250000   8.167438867622e-02   6.9345671125e-04   3.7500000000e-03
  9   0.4471875000  -2.823744058128e-02   2.4404328875e-04   1.8750000000e-03
 10   0.4476562500   2.623077527038e-02   2.2470671125e-04   9.3750000000e-04
Bisection final approx: 0.447656250000,  |err| = 2.247e-04

=== Secant (N = 10) starting p0=0.000000, p1=0.480000 ===
  k            p_k               f(p_k)      |p_k - alpha|
  1   0.1811942417  -5.360105281562e+00   2.6623730160e-01
  2   0.2861871658  -4.742210953661e+00   1.6124437747e-01
  3   1.0919861065  -5.702694576330e+00   6.4455456321e-01
  4  -3.6922966654  -4.551142530387e+00   4.1397282087e+00
  5 -22.6006498547  -2.943562686809e+00   2.3048081398e+01
  6 -57.2228324726  -6.842371905747e+00   5.7670264016e+01
  7   3.5387581457  -1.417209464998e+01   3.0913266024e+00
  8 -113.9444050481  -5.823545401098e+00   1.1439183659e+02
  9 -195.8949948245  -5.657605308972e+00   1.9634242637e+02
 10 -2989.9400375314  -5.809361994173e+00   2.9903874691e+03
Secant final approx:   -2989.9400375314445,  |err| = 2.990e+03

=== Verdict ===
True root alpha ~= 0.447431543289
Bisection (robust bracket) reached |err| = 2.247e-04 after 10 steps.
```

Most successful: Bisection.
Why: It is a bracketing method. Since $f(0) = -6 < 0$ and $f(0.48) > 0$, continuity on $[0, 0.48]$ guarantees a root inside and **guaranteed convergence** with **linear rate**. After n steps, the error is bounded by $\frac{b-a}{2^n}$ (here $\frac{0.48}{2^{10}} \approx 4.69 \times 10^{-4}$), consistent with the 2.247×10^{-4} observed error.

Why secant failed here: Secant is faster **when** it converges, but it **does not preserve a bracket** and can **diverge** if the initial points are not close enough or the function is poorly behaved. The function $f(x) = \tan(\pi x) - 6$ has a **vertical asymptote at** $x = 0.5$ (since $\tan z$ blows up at $z = \pi/2 + k\pi$), so the slope near the root is extremely steep. With starting points straddling a region close to the asymptote, secant steps become unstable, which matches the exploding iterates.

18

MATLAB codes:

```
function newton_demo
    % Define f and f' (two equivalent ways; pick either block).

    % --- Direct trigonometric form ---
    f  = @(x) 0.5 + 0.25*x.^2 - x.*sin(x) - 0.5*cos(2*x);
    fp = @(x) 0.5*x - sin(x) - x.*cos(x) + sin(2*x);

    % % --- Perfect-square form (numerically nice) ---
    % f  = @(x) (0.5*x - sin(x)).^2;
    % fp = @(x) 2*(0.5*x - sin(x)).*(0.5 - cos(x));

    tol   = 1e-5;
    maxit = 100;

    p0s = [pi/2, 5*pi, 10*pi];
    for p0 = p0s
        [p, k, flag] = newton(f, fp, p0, tol, maxit);
        if flag == 0
            fprintf('p0 = %12.8f  ->  p = %12.8f in %2d iters,  f(p) = %.3e\n', ...
                    p0, p, k, f(p));
        else
            fprintf(['p0 = %12.8f  ->  FAILED after %d iterations; ' ...
                     'last p = %.8g, f(p) = %.3e\n'], ...
                    p0, k, p, f(p));
        end
    end
end

function [p, k, flag] = newton(f, fp, p0, tol, maxit)
    % Basic Newton iteration with |p_{n+1}-p_n| stopping criterion
    p = p0; k = 0; flag = 1;
    while k < maxit
        df = fp(p);
        if df == 0, return; end       % avoid division by zero
        p_new = p - f(p)/df;
        k = k + 1;
        if abs(p_new - p) < tol
            p = p_new; flag = 0; return;
        end
        p = p_new;
    end
end
```

Outputs:

```
p0 =   1.57079633  ->  p =   1.89548842 in 15 iters,  f(p) = 2.294e-11
p0 =  15.70796327  ->  p =   1.89548900 in 19 iters,  f(p) = 1.860e-11
p0 =  31.41592654  ->  FAILED after 100 iterations; last p = 286041.46, f(p) = 2.046e+10
```

What the iterations give (to 10^{-5})

- $p_0 = \frac{\pi}{2}$:
  Newton on f converges linearly to
  $\boxed{x^\star \approx 1.89549}$ (1.89548842 after 15 iters).
- $p_0 = 5\pi$:
  Newton on f again converges linearly to the **same** root
  $\boxed{x^\star \approx 1.89549}$ (1.89548900 after 19 iters).
- $p_0 = 10\pi$:
  Newton on f **fails** (derivative near zero creates huge steps → blow-up; the last iterate is $\approx 2.86 \times 10^5$).

Why this looks "unusual" for Newton

$$f(x) = \tfrac{1}{2} + \tfrac{1}{4}x^2 - x\sin x - \tfrac{1}{2}\cos(2x) = \tfrac{1}{4}x^2 - x\sin x + \sin^2 x = \left(\tfrac{x}{2} - \sin x\right)^2$$

Every zero of f is a double root (multiplicity $m = 2$). Newton's method is only **linear** near multiple roots; the error contracts roughly by a factor $(m-1)/m = 1/2$ each step, instead of quadratically. That is exactly the slowdown observed (15–19 steps instead of a handful).

30

The two forms of the iteration equation for the Secant method are algebraically equivalent, as can be verified by rearranging one into the other. Start with the form from Algorithm 2.4:

$$p_n = p_{n-1} - f(p_{n-1})\frac{p_{n-1} - p_{n-2}}{f(p_{n-1}) - f(p_{n-2})}.$$

To obtain the given form, rewrite the expression over a common denominator:

$$p_n = \frac{p_{n-1}[f(p_{n-1}) - f(p_{n-2})] - f(p_{n-1})(p_{n-1} - p_{n-2})}{f(p_{n-1}) - f(p_{n-2})} = \frac{p_{n-1}f(p_{n-1}) - p_{n-1}f(p_{n-2}) - f(p_{n-1})p_{n-1} + f(p_{n-1})p_{n-2}}{f(p_{n-1}) - f(p_{n-2})} = \frac{f(p_{n-1})p_{n-2} - f(p_{n-2})p_{n-1}}{f(p_{n-1}) - f(p_{n-2})}.$$

However, the forms are not numerically equivalent in floating-point arithmetic due to the potential for subtractive cancellation in the given form. As the method converges and p_{n-1} approaches p_{n-2}, the terms $f(p_{n-1})p_{n-2}$ and $f(p_{n-2})p_{n-1}$ in the numerator become nearly equal, both approximately $f(p_{n-2})p_{n-2}$. If $|f(p_{n-2})|$ and $|p_{n-2}|$ are large in magnitude, these are two large and close numbers whose difference is small, leading to a loss of significant digits in the numerator. This reduces the overall accuracy of p_n.

In contrast, the form in Algorithm 2.4 computes a small correction term $\frac{p_{n-1}-p_{n-2}}{f(p_{n-1})-f(p_{n-2})}$ (a ratio that may involve small numbers in both numerator and denominator but without requiring subtraction of large close values) multiplied by $f(p_{n-1})$ and subtracted from p_{n-1}. Subtracting a small correction from a large p_{n-1} does not cause catastrophic cancellation, preserving more significant digits. Thus, the form in Algorithm 2.4 is generally more numerically stable.

Problem 2: Problems from the textbook, chapter 2.4

1b

MATLAB codes:

```
%% Newton's method on cos(x+sqrt(2)) + x*(x/2 + sqrt(2)) = 0, x in [-2, -1]
% Target accuracy: 1e-5 (in x)
% This script tries a few starting points in [-2, -1] and reports unique roots.

clear; clc;

% --- problem setup ---
f  = @(x) cos(x + sqrt(2)) + x.*(x/2 + sqrt(2));
fp = @(x) -sin(x + sqrt(2)) + x + sqrt(2);   % derivative

tol   = 1e-5;      % stop when |x_{k+1} - x_k| < tol
maxit = 100;       % safety cap on iterations

% choose several starting points in the interval
starts = linspace(-2, -1, 5);

roots_found = [];
fprintf('Newton iterations (tol = %.1e):\n', tol);
for s = starts
    [root, iters, ok] = newton_scalar(f, fp, s, tol, maxit);
    if ok
        % keep unique roots only (up to tol)
        if isempty(roots_found) || all(abs(roots_found - root) > 5*tol)
            roots_found(end+1) = root; %#ok<AGROW>
        end
        fprintf('  start = %+ .8f -> root = %+ .10f  (iters = %d)\n', s, root, iters);
    else
        fprintf('  start = %+ .8f -> FAILED to converge in %d iters\n', s, maxit);
    end
end

% --- report ---
if isempty(roots_found)
    fprintf('\nNo roots found in [-2, -1] with these initial guesses.\n');
else
    fprintf('\nUnique root(s) found in [-2, -1]:\n');
    for r = roots_found
        fprintf('  x ≈ %+ .10f   |f(x)| = %.3e\n', r, abs(f(r)));
    end
end

% --- compare to the exact root x = -sqrt(2) ---
x_exact = -sqrt(2);
fprintf('\nExact root check: x = -sqrt(2) ≈ %+ .10f,  f(x) = %.3e\n', x_exact, f(x_exact));

% ============================
% Newton method (scalar) ---
% ============================
function [x, k, success] = newton_scalar(f, fp, x0, tol, maxit)
%NEWTON_SCALAR  Basic Newton's method for a scalar nonlinear equation.
%   Inputs:
%     f, fp  - function handle and its derivative
%     x0     - initial guess
%     tol    - tolerance on |x_{k+1} - x_k|
%     maxit  - maximum iterations
%   Outputs:
%     x        - final iterate
%     k        - iterations used
%     success  - true if converged within maxit
```

```
    x = x0;
    success = false;
    for k = 1:maxit
        fx  = f(x);
        fpx = fp(x);
        if ~isfinite(fx) || ~isfinite(fpx) || fpx == 0
            % derivative zero or NaN/Inf — abort this start
            success = false;
            return;
        end
        xnew = x - fx/fpx;
        if ~isfinite(xnew)
            success = false;
            return;
        end
        if abs(xnew - x) < tol
            x = xnew;
            success = true;
            return;
        end
        x = xnew;
    end
end
```

Outputs:

```
Newton iterations (tol = 1.0e-05):
  start = -2.00000000 -> root = -1.4144439327  (iters = 29)
  start = -1.75000000 -> root = -1.4144632112  (iters = 27)
  start = -1.50000000 -> root = -1.4144727378  (iters = 22)
  start = -1.25000000 -> root = -1.4139642121  (iters = 24)
  start = -1.00000000 -> root = -1.4139553890  (iters = 27)

Unique root(s) found in [-2, -1]:
  x ≈ -1.4144439327   |f(x)| = 0.000e+00
  x ≈ -1.4139642121   |f(x)| = 0.000e+00

Exact root check: x = -sqrt(2) ≈ -1.4142135624,  f(x) = -2.220e-16
```

6b

Compute the Definition-2.7 ratio with $\alpha = 1$:

$$\frac{|p_{n+1} - p|}{|p_n - p|^{\alpha}} = \frac{\frac{1}{(n+1)^2}}{\frac{1}{n^2}} = \left(\frac{n}{n+1}\right)^2 \xrightarrow[n\to\infty]{} 1.$$

Hence the sequence converges to $p = 0$ **with order** $\alpha = 1$ and **asymptotic error constant** $\lambda = 1$. By standard terminology, "linear convergence" requires $\alpha = 1$ **and** $0 < \lambda < 1$; when $\lambda = 1$ it is *order 1 but not (strictly) linear*.

$$|p_n - p| = \frac{1}{n^2} \le 5 \times 10^{-2} \quad \Longleftrightarrow \quad n^2 \ge 20 \quad \Longleftrightarrow \quad n \ge \lceil \sqrt{20} \rceil = 5.$$

10

Let p be a zero of multiplicity m of f. Then $f(x)$ can be written as

$$f(x) = (x - p)^m q(x), \qquad q(p) \neq 0,$$

and hence

$$f'(x) = m(x - p)^{m-1} q(x) + (x - p)^m q'(x) = (x - p)^{m-1}[m q(x) + (x - p) q'(x)].$$

This representation is the standard characterization of a multiple root.

Now consider

$$g(x) = x - \frac{m f(x)}{f'(x)} = x - \frac{m(x - p)^m q(x)}{(x - p)^{m-1}[m q(x) + (x - p) q'(x)]} = x - \frac{m(x - p) q(x)}{m q(x) + (x - p) q'(x)}.$$

Subtract p and simplify:

$$g(x) - p = (x - p)\left[1 - \frac{m q(x)}{m q(x) + (x - p) q'(x)}\right] = (x - p)\frac{(x - p) q'(x)}{m q(x) + (x - p) q'(x)} = \frac{q'(x)}{m q(x) + (x - p) q'(x)}(x - p)^2.$$

Hence

$$\frac{g(x)-p}{x-p} = \frac{q'(x)}{mq(x)+(x-p)q'(x)}(x-p) \xrightarrow[x\to p]{} 0,$$

because the denominator tends to $mq(p) \neq 0$. Therefore

$$g'(p) = \lim_{x\to p} \frac{g(x)-p}{x-p} = 0.$$

```
Dataview (inline field '== Newton's Method (TOL = 1e-05) ===

(a) [1,2]   root ≈ 1.8293836019   residual f(x) = 8.882e-16
(b) [1.3,2] root ≈ 1.3977484759   residual f(x) = -1.006e-10
(c) [2,3]   root ≈ 2.3706869177   residual f(x) = 2.248e-12
(c) [3,4]   root ≈ 3.7221127731   residual f(x) = -7.867e-11
(d) [1,2]   root ≈ 1.4123911720   residual f(x) = 1.665e-16
(d) [e,4]   root ≈ 3.0571035500   residual f(x) = 2.154e-13
(e) [0,1]   root ≈ 0.9100075725   residual f(x) = -1.377e-14
(e) [3,5]   root ≈ 3.7330790287   residual f(x) = 4.245e-10
(f) [0,1]   root ≈ 0.5885327440   residual f(x) = -6.161e-12
(f) [3,4]   root ≈ 3.0963639324   residual f(x) = -1.041e-16
(f) [6,7]   root ≈ 6.2850492734   residual f(x) = 1.427e-16

Done.'): Error:
-- PARSING FAILED -----------------------------------------------

> 1 | == Newton's Method (TOL = 1e-05) ===
    | ^
  2 |
  3 | (a) [1,2]   root ≈ 1.8293836019   residual f(x) = 8.882e-16

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string,
variable



Dataview (inline field '== Secant Method (TOL = 1e-05) ===

(a) [1,2]   root ≈ 1.8293836019   residual f(x) = 2.307e-11
(b) [1.3,2] root ≈ 1.3977484760   residual f(x) = -2.377e-12
(c) [2,3]   root ≈ 2.3706869177   residual f(x) = 3.376e-13
(c) [3,4]   root ≈ 3.7221127734   residual f(x) = -5.195e-09
(d) [1,2]   root ≈ 1.4123911720   residual f(x) = -1.742e-13
(d) [e,4]   root ≈ 3.0571035500   residual f(x) = 6.802e-11
(e) [0,1]   root ≈ 0.9100075715   residual f(x) = 2.827e-09
(e) [3,5]   root ≈ 3.7330790313   residual f(x) = 5.217e-08
(f) [0,1]   root ≈ 0.5885327423   residual f(x) = -2.266e-09
(f) [3,4]   root ≈ 3.0963639324   residual f(x) = -1.993e-11
(f) [6,7]   root ≈ 6.2850492734   residual f(x) = 2.226e-13

Done.'): Error:
-- PARSING FAILED -----------------------------------------------

> 1 | == Secant Method (TOL = 1e-05) ===
    | ^
  2 |
  3 | (a) [1,2]   root ≈ 1.8293836019   residual f(x) = 2.307e-11

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string,
variable



Dataview (inline field '== Bisection (N = 10) on [0.000000, 0.480000] ===
  k            c_k                f(c_k)       |c_k - alpha|     bracket length
  1   0.2400000000  -5.060937494183e+00   2.0743154329e-01   4.8000000000e-01
  2   0.3600000000  -3.874891826843e+00   8.7431543289e-02   2.4000000000e-01
  3   0.4200000000  -2.105257145070e+00   2.7431543289e-02   1.2000000000e-01
  4   0.4500000000   3.137515146750e-01   2.5684567113e-03   6.0000000000e-02
  5   0.4350000000  -1.171182647807e+00   1.2431543289e-02   3.0000000000e-02
  6   0.4425000000  -5.245211508219e-01   4.9315432887e-03   1.5000000000e-02
  7   0.4462500000  -1.343497628774e-01   1.1815432887e-03   7.5000000000e-03
  8   0.4481250000   8.167438867622e-02   6.9345671125e-04   3.7500000000e-03
  9   0.4471875000  -2.823744058128e-02   2.4404328875e-04   1.8750000000e-03
```

```
 10    0.4476562500    2.623077527038e-02    2.2470671125e-04    9.3750000000e-04
Bisection final approx: 0.447656250000,  |err| = 2.247e-04

=== Secant (N = 10) starting p0=0.000000, p1=0.480000 ===
  k             p_k                f(p_k)       |p_k - alpha|
  1    0.1811942417   -5.360105281562e+00    2.6623730160e-01
  2    0.2861871658   -4.742210953661e+00    1.6124437747e-01
  3    1.0919861065   -5.702694576330e+00    6.4455456321e-01
  4   -3.6922966654   -4.551142530387e+00    4.1397282087e+00
  5  -22.6006498547   -2.943562686809e+00    2.3048081398e+01
  6  -57.2228324726   -6.842371905747e+00    5.7670264016e+01
  7    3.5387581457   -1.417209464998e+01    3.0913266024e+00
  8 -113.9444050481   -5.823545401098e+00    1.1439183659e+02
  9 -195.8949948245   -5.657605308972e+00    1.9634242637e+02
 10 -2989.9400375314   -5.809361994173e+00    2.9903874691e+03
Secant final approx:   -2989.940037531445,  |err| = 2.990e+03

=== Verdict ===
True root alpha ~= 0.447431543289
Bisection (robust bracket) reached |err| = 2.247e-04 after 10 steps.'): Error:
-- PARSING FAILED -----------------------------------------------

> 1 | == Bisection (N = 10) on [0.000000, 0.480000] ===
    | ^
  2 |   k            c_k                f(c_k)       |c_k - alpha|     bracket length
  3 |   1    0.2400000000   -5.060937494183e+00    2.0743154329e-01    4.8000000000e-01

Expected one of the following:

'(', 'null', boolean, date, duration, file link, list ('[1, 2, 3]'), negated field, number, object ('{ a: 1, b: 2 }'), string,
variable
```